A 28-nm 19.9-to-258.5-TOPS/W 8b Digital Computing-in-Memory Processor With Two-Cycle Macro Featuring Winograd-Domain Convolution and Macro-Level Parallel Dual-Side Sparsity

Hao Wu, *Member, IEEE*, Yong Chen, *Senior Member, IEEE*, Yiyang Yuan, Jinshan Yue, *Member, IEEE*, Xinghua Wang, *Member, IEEE*, Xiaoran Li, *Member, IEEE*, and Feng Zhang, *Senior Member, IEEE*

***Abstract*— Recently, computing in memory (CiM) has been proven to be an energy-efficient and promising architecture for artificial intelligence (AI) algorithms. And yet, current CiM schemes generally suffer from limited *throughput* compared to their digital counterparts, and the key reason is that the CiM macro calculation must iterate through multiple cycles. Thus, the need to reduce the calculation cycle of the macro while keeping high energy efficiency and the necessity of developing acceleration methods for the universal CiM-based processor have become major issues faced by the current CiM architectures. To surmount these critical problems, we propose a processor based on a two-cycle CiM macro. Our work makes three main contributions: 1) we present a Radix16-based digital-CiM macro with look-up table (LUT) optimization to reduce dynamic power consumption; 2) we devise a hybrid Winograd microarchitecture and dataflow that supports (2, 3) and (4, 3) Winograd convolution, meaning that a good compromise can be reached between the accuracy of the algorithm and the reduction in workload; and 3) we propose a macrolevel parallel dual-side sparse CiM core that uses a horizontal direction compression method to reduce the input cycle of activation data and improve the mapping efficiency of the weight data in the macros. A prototype of the processor is fabricated in a 28-nm CMOS, which achieves a peak system energy efficiency of 19.9–258.5-TOPS/W for a voltage supply of 0.6–1.1 V, and an operating frequency of 78–287 MHz, a 2.55–7.08× higher than other state-of-the-art CiM processors.**

***Index Terms*— Artificial intelligence (AI), CMOS, computing-in-memory (CiM), energy efficiency, look-up table (LUT), multiply-accumulation (MAC), neural network (NN), Radix16, unstructured sparsity, Winograd convolution.**

Manuscript received 6 December 2023; revised 28 February 2024 and 14 May 2024; accepted 25 May 2024. Date of publication 18 June 2024; date of current version 20 January 2025. This article was approved by Associate Editor Meng-Fan Chang. This work was supported in part by the National Key Research Plan of China under Grant 2023YFB4402400, in part by the Strategic Priority Research Program of the Chinese Academy of Sciences China under Grant XDB44000000 and Grant XDA330100, in part by the National Natural Science Foundation of China under Grant U2341218 and Grant 62101038, and in part by the Science and Technology Development Fund (FDCT) under Grant 0036/2020/AGJ and Grant MYRG2020-00209-IME. *(Corresponding authors: Yong Chen; Feng Zhang.)*

Hao Wu, Yiyang Yuan, Jinshan Yue, and Feng Zhang are with the Key Laboratory of Microelectronics Device and Integrated Technology, Institute of Microelectronics, Chinese Academy of Sciences (CAS), Beijing 100029, China, and also with the School of Integrated Circuit, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: yuejinshan@ime.ac.cn; zhangfeng_ime@ime.ac.cn).

Yong Chen was with the State-Key Laboratory of Analog and Mixed-Signal VLSI and IME/ECE-FST, University of Macau, Macau, China. He is now with the Department of Electronic Engineering, Tsinghua University, Beijing 100190, China (e-mail: ychen_ee@tsinghua.edu.cn).

Xinghua Wang and Xiaoran Li are with the School of Information and Electronics, Beijing Institute of Technology, Beijing 100081, China.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSSC.2024.3409356.

Digital Object Identifier 10.1109/JSSC.2024.3409356

I. Introduction

FOLLOWING the emergence of large universal models such as ChatGPT, the growing computing loads of artificial intelligence (AI) algorithms have imposed higher requirements on silicon-based hardware. Traditional digital AI accelerators based on the von Neumann architecture have achieved good system energy efficiency. These accelerators are based on separate memory and compute arrays for handling various AI workloads [1], [2], [3], [4], [5], [6], as shown in Fig. 1. However, it is hard to further improve the energy efficiency of AI computing, since the problem of the high power required for massive data transmission cannot be solved [7].

Computing in memory (CiM) is the most promising candidate in terms of breaking through the memory wall bottleneck. In this approach, storage units and computing circuits are coupled together to reduce the power consumption required for large data transfers. The current CiM implementation uses a specially designed analog-to-digital converter (ADC) to read out the multiply-accumulation (MAC) calculation results over multiple cycles (i.e., analog CiM), or decomposes the multiplication operation into several steps to obtain the MAC calculation results over multiple cycles (i.e., digital CiM), and both of these approaches have yielded good energy efficiency [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20].

However, there are still some problems with this coupling scheme. Compared to the traditional digital pipeline structure, the throughput is greatly reduced by the special calculation method used, which consumes multiple CiM calculation cycles to deliver the complete MAC calculation results. This means that the computational throughput of CiM-based processors is completely determined by the number of computational cycles, for a given operating frequency and a given number of MAC units. As shown in Fig. 1, it is hard to increase the

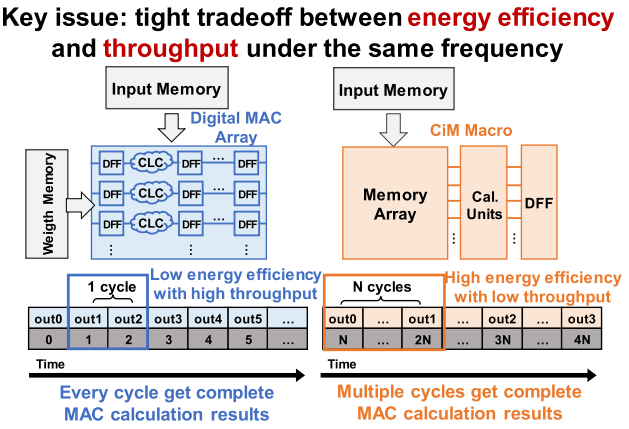


Fig. 1. Low throughput as a bottleneck for current CiM architectures.

throughput of the current CiM architecture while maintaining high-energy efficiency, and this architecture is therefore limited to low-power scenarios rather than high-performance ones. To allow us to address this key issue, we summarize the existing challenges at three different levels as follows.

Challenge 1: At the circuit level, for the digital CiM macro, a method is needed to reduce the number of the CiM calculation cycles while retaining the advantage of the CiM's high-energy efficiency.

Challenge 2: At the microarchitecture level, a dataflow and microarchitecture are required to develop a universal Winograd optimization to support various configurations of convolutional calculations. It is also necessary to solve the problem of the reduced accuracy of the algorithm arising from the use of the Winograd method.

Challenge 3: At the data level, sparsity has always offered an important opportunity for the optimization of AI accelerators. A hardware architecture is required that can support flexible skipping of unnecessary calculations to reduce power consumption and improve throughput. These challenges are illustrated in Fig. 2.

In this article, we propose a digital CiM-based high-throughput processor with a two-cycle macro based on Winograd-domain convolution (CONV) and macrolevel parallel dual-side sparsity. Our work makes three main contributions to this.

To address *Challenge 1*, a new digital CiM macro is proposed that can reduce the number of CiM calculation cycles to two for INT8. Concurrently, a look-up table (LUT) method is introduced that can reduce the dynamic power consumption of the digital CiM macro by 21.7%. The macrolevel energy efficiency ratio can be improved by 1.84–2.44×.

In response to *Challenge 2*, we develop two different dataflow modes: a Winograd-domain data path is used to support Winograd CONV calculations of F2 and F4, and a spatial-domain data path executes 1 × 1 CONV calculations and general matrix-to-matrix multiplication (GEMM) calculations in the fully connected (FC) layer. The accuracy of the algorithm is maintained while the number of MAC operations is reduced as far as possible, and an acceleration of 2.59× is achieved.

To tackle *Challenge 3*, we devise a macrolevel dual-side parallel sparsity (MDPS) strategy to realize sparsity optimization without loss of accuracy of the algorithm. The number of input cycles for the activation tiles is reduced, while the mapping efficiency of weight data on the CiM macro is increased. A speedup of 3.11× is attained upon both operations having 50% sparsity.

II. Background and Motivation

A. Evolution and Key Issues of CiM Architecture

The CiM architecture has proven to be a very promising choice for AI workloads. The weight data for different AI workloads are stored in a CiM macro and directly used for local calculations to reduce the weight data transfer cost of the AI workload. Among the current approaches, digital CiM has better scaling capability than analog CiM, and improvements in energy efficiency can be made with advancements in chip process nodes. This approach solves the issue of nonideal factors caused by the processing of analog signals in analog CiM. A digital CiM macro can activate multiple rows at the same time, and a single macro serves as multiple inner product calculation units for the implementation of GEMM in AI workloads. Hence, digital CiM architecture is expected to become a mainstream option for the next generation of AI accelerators. However, its throughput depends strongly on the number of calculation cycles of the CiM macro circuit, meaning that a reduction in the number of cycles can make a huge improvement to the throughput.

The earliest implementation of digital CiM was a bit-serial scheme that could achieve high computing energy efficiency but only obtained the complete MAC result every eight cycles for INT8. Researchers have since developed several methods of reducing the number of CiM calculation cycles (Fig. 3). Wen et al. [20] used a threshold judgment to decide whether to flexibly skip the calculation of some input bits in analog CiM, which could reduce the number of CiM calculation cycles but caused a loss of accuracy. The study in [9] presented a duplicated adder tree (AT) to cope with multiple input bits within one cycle. The number of calculation cycles was reduced, but the energy efficiency was not significantly improved because the duplicated AT introduced more circuit flips per cycle. In [15], Booth coding was employed to handle multiplication calculations, which reduced the number of calculation cycles and improved the energy efficiency; yet, INT8 still required four cycles to obtain the complete MAC result. The issue of how to simultaneously reduce the number of MAC calculation cycles while minimizing the introduction of additional circuit flipping is a key issue currently faced by designers in this field.

B. Winograd-Domain CONV

The convolutional neural network (CNN) is a kernel part of AI workloads, and many methods have been developed to speed up the CONV calculations. The Winograd algorithm is an extended Toom–Cook algorithm that combines cheap transformation operations with constant matrices to minimize the number of multiplications required in the CONV calculation [21], [22]. The extra transformation operation has a lower

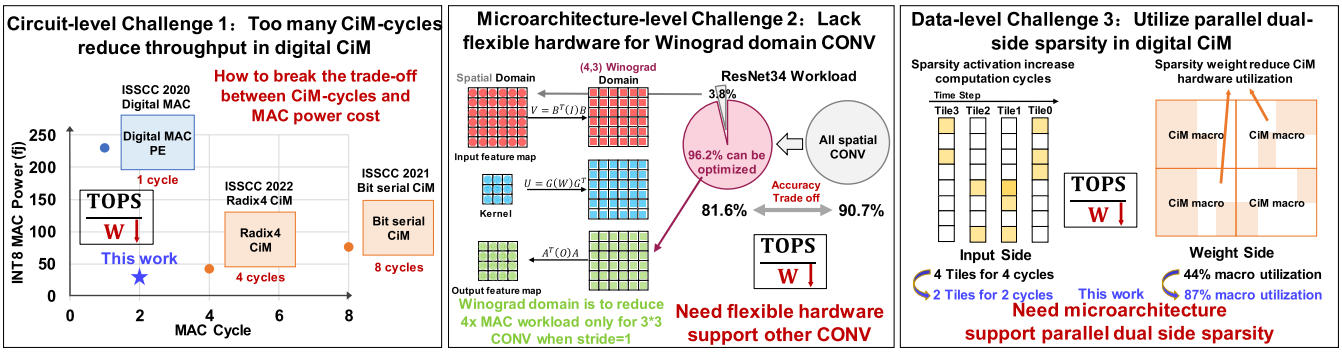


Fig. 2. Challenges at different levels identified in prior works aiming to reduce the number of calculation cycles.

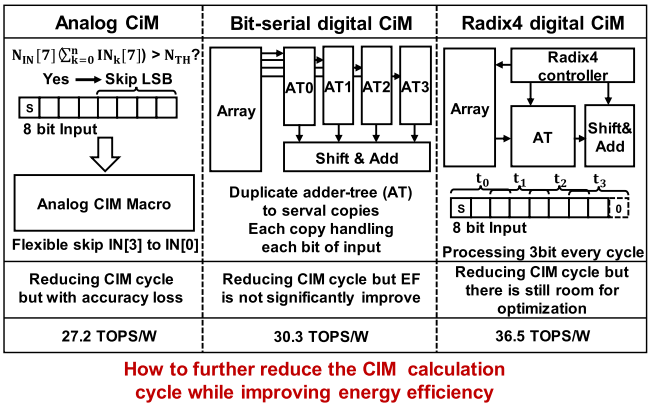


Fig. 3. Methods used in previous CiM variants to reduce the number of calculation cycles.

TABLE I
TRADITIONAL/WINOGRAD CONV ACCURACY IN 8 bit

Task	ResNet18 Cifar10 Top-1		
Traditional CONV	93.22%		
Winograd CONV	(2,3)	(4,3)	(6,3)
Accuracy without Winograd-Aware	93.21%	17.36%	10.95%
Accuracy with Winograd-Aware	92.13%	80.25%	81.34%
Speedup for 3x3 CONV	2.25x	4x	5.06x

power cost than the multiplication in the CONV workload. There are different levels of the Winograd algorithm. The Winograd algorithm for a 3 × 3 CONV with a single output tile size of $m \times m$ is represented by F(m, 3). The most common ones are F(2, 3), F(4, 3), and F(6, 3), also denoted as F2, F4, and F6. In terms of floating point precision, the Winograd algorithm can achieve a 2.24–5.06× reduction in MAC workload without loss of accuracy. Currently, to accelerate AI inference workloads, data accuracy is generally limited to INT8. However, with low data accuracy, the Winograd optimization method will cause irreparable accuracy loss. As shown in Table I, using the F4/F6 optimization method directly under conditions of low data precision will lead to a drastic decrease in accuracy [23]. To enable this optimization method to be integrated into the CiM chip, the hardware design requires a flexible data path to support both the spatial-domain and the Winograd-domain CNN workload. A microarchitecture is needed that allows the Winograd optimization method to support CONV calculations with different strides and different kernel sizes.

C. Sparsity Optimization

Sparsity is an essential feature of neural network (NN) algorithms. Prior researchers [24] have proved that NN algorithms can be compressed to 95% sparsity without accuracy loss, since due to the quantization operations and activation functions involved, a large proportion of the weights and activation data of the NN will be zero-valued after processing. The MAC calculations of these zero-value data will not affect the results of the NN algorithm, which gives rise to opportunities for optimization and hardware acceleration of the algorithm. As summarized in Fig. 4, recent sparsity technology was implemented using CiM macro at the hardware level for different types of AI workloads. At the level of sparsity, these optimization methods can be divided into three categories: blockwise [11], [12], elementwise [13], [16], [17], [18], [31], [32], bitwise sparsity methods [14], [17], [19]. Processors in [16] and [17] proposed sparsity optimization techniques specifically for transformers. Yue et al. [13] reported a sparsity optimization technique specific to graph neural network (GCN)/deep learning recommendation models (DLRMs). These technologies introduce very small hardware and power consumption overhead and exhibit good energy efficiency improvements. However, these techniques are only applicable to these specific algorithms and are not generally the sparsity techniques for AI computing. Yue et al. [14] and Zhu et al. [19] used the innovation point 3 of [17] designs bitwise general sparsity optimization technology. Yue et al. [14] and Zhu et al. [19] reduced the power consumption of the readout-sense amplifier (SA) circuit of the static random access memory (SRAM) bit-cell. Yet, in the digital CiM macro, the power consumption of logic circuits, such as adders, which account for a higher proportion of power consumption, has not been reduced. Bitwise sparsity optimization in [14] and [19] does not gain performance by profiting from sparse data (e.g., the calculation of the sparse bits is not skipped). The sparsity optimization technology in [17] is only applicable to bit-serial CiM macro. Tu et al. [18] exploited elementwise sparsity technology using digital core

and CiM core to handle sparse and dense data, respectively. Since the proportion of sparse data is unknown, there may be an imbalance in the computational load between different types of cores. Yue et al. [11] and [12] presented structured sparsity optimization methods. Through a special training method, the weight data are sparse in units of data blocks. This allows weight data to be mapped more easily into the CiM macro's rigid array. But the sparse weight data distribution in general trained algorithms is random, thus the universality of this method is still affected by changes in training methods. Customized training methods need to be designed for each different AI workload such as CNN, transformer, and LSTM. In most algorithms, the sparsity of weight data is randomly distributed because they use general training methods. Therefore, the performance gains of unstructured sparse data cannot be implemented by using this type of hardware microarchitecture. Yue et al. [12] used a method to adjust the accuracy of an ADC's readout results to handle sparse activation data. This method is only suitable for analog CiM macro but is implemented at the cost of algorithm accuracy. Both the weight data compression methods in [31], [32], and [33], as well as the column/row-exchange method in [32], require weight data to adhere to specific formats. Most importantly, these two sparsity methods only apply to macros that activate the operation unit (OU) serially, which greatly reduces computing parallelism and the data reuse rate.

In summary, we need a universal sparsity technique for digital CiM macros that can handle both activation/weight dual-side sparse data. This sparsity optimization method should be able to achieve the targets of both power consumption reduction and performance improvement and maintain excellent universality for different AI workloads.

III. Overall Architecture of the Proposed CiM Processor

Fig. 5(a) shows the overall architecture of the proposed general purpose digital CiM processor and its three features. The entire CiM processor incorporates six processing cores, a 130.5 kB global SRAM, a 121.5 kB weight SRAM, and an RISC-V-based top control unit with a 4 kB inst SRAM. Each processing core contains a 144 × 32 CiM core, an activation input buffer, three Winograd transformation units, a single instruction multiple data (SIMD) unit, and a 6 kB configuration information SRAM. Each CiM core contains four 36 × 32 CiM macros, an input sparsity-processing unit (ISPU), and an output merge buffer (OMB).

As shown in Fig. 5(b), the entire chip has two work modes. In the Winograd-domain mode, the activation data are first read from the global SRAM and are sent to the input buffer, which cooperates with the $B^T()B$ unit to complete the Winograd transformation of the activation data. Since the bit width of the transformed data will change, the data needs to be quantized into 8 bits by the SIMD unit after the transformation operation, before being sent to the CiM core. The weight data are read from the weight SRAM and are passed to the CiM core for calculation after the $G()G^T$ transformation and quantization operations for reducing weight SRAM capacity. The MAC result obtained by the CiM core needs to be transformed by the $A^T()A$ unit, and after being processed by the SIMD unit, it is quantized into 8-bit blocks for subsequent calculations. In summary, in the Winograd-domain mode, three transformation units are responsible for converting the data between the Winograd and spatial domains. The Winograd-domain mode is used for the CONV workload optimized by the Winograd method. In the spatial-domain mode, the three transformation units do not need to be enabled, as the CiM core and SIMD unit perform regular MAC calculations. The spatial-domain mode is designed to process the spatial-domain CONV workload and the GEMM workload at the FC layer.

IV. Radix16 + LUT-Based Digital CiM Macro for Circuit-Level Optimization

A. Radix16 + LUT Mechanism

The CiM macro circuit is a key component of the CiM-based processor. To increase the processor throughput while maintaining high-energy efficiency, it is extremely important to reduce the number of calculation cycles of the CiM macro. In this section, we first introduce the characteristics of the typical architecture of a digital CiM macro. The implementation formulae of the two different multiplications for INT n are as follows:

$$I * W = \sum_{i=0}^{n-1} \left(I[i] * W * 2^i \right) \tag{1}$$

$$I * W = \sum_{i=0, i+=2}^{n-1} \left((-2I[i+1] + I[i] + I[i-1]) * W * 2^i \right). \tag{2}$$

As can be seen from the above formulae, the bit-serial multiplication in (1) involves processing 1 bit of activation data per calculation cycle [8]. This is the most direct method of multiplication splitting and can achieve high-energy efficiency, but the processing of only 1 bit of activation data at a time makes this method inefficient in the time domain. Tu et al. [15] introduced the Radix4-based multiplication in (2), which can compute 3 bits of activation data at a time. This improves the processing efficiency in the time domain and reduces the CiM calculation cycles by half, but is still very slow to complete the INT8 × INT8 multiplication every four cycles. To reduce the number of calculation cycles for this operation, there is a need to increase the circuit flips per cycle, which worsens the energy efficiency. Thus, it is necessary to reduce the number of calculation cycles while minimizing the extra circuit flips.

To address these issues, we propose a Radix16 + LUT method for the digital CiM macro in our processor. This multiplication can be written as

$$I * W = \sum_{i=0, i+=4}^{n-1} \left((-8I[i+3] + 4I[i+2] + 2I[i+1] + I[i] + I[i-1]) * W * 2^i \right). \tag{3}$$

Unlike the methods in [8] and [15], this method processes 5 bits of activation data each time and can complete the INT8 × INT8 multiplication in two calculation cycles. However, the circuit that directly implements the weighted partial product calculation will introduce too many circuit flips in each calculation cycle. For this reason, we introduce the LUT

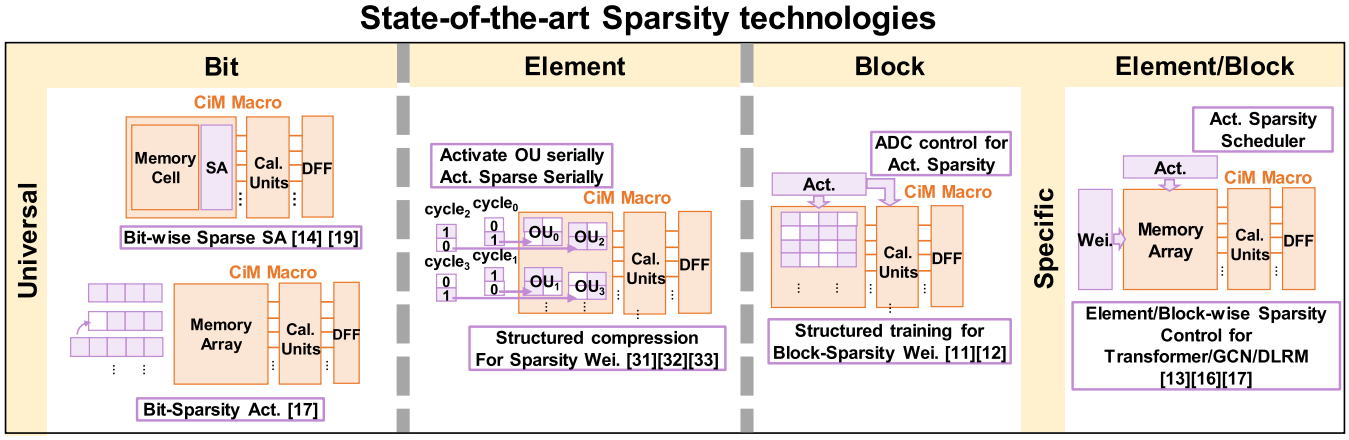


Fig. 4. Classification of sparsity techniques in CiM-based processors from previous work.

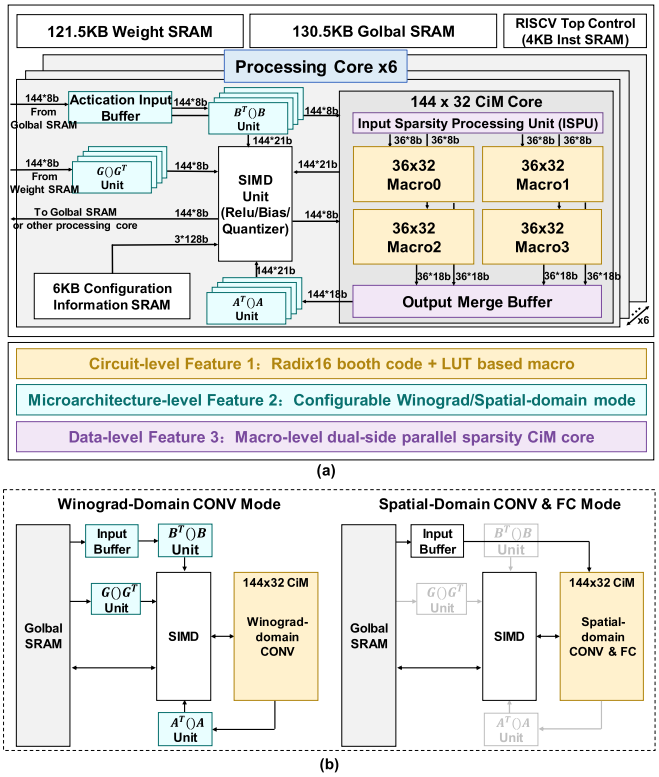
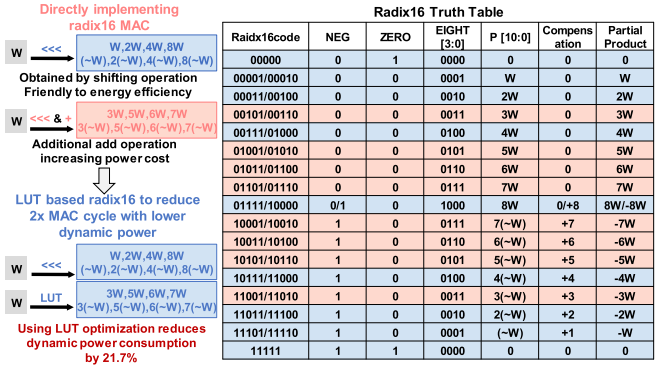


Fig. 5. (a) Proposed top-level architecture with three features and (b) dataflow direction for the two modes of operation.



Raidx16code	NEG	ZERO	EIGHT [3:0]	P [10:0]	Compensation	Partial Product
00000	0	1	0000	0	0	0
00001/00010	0	0	0001	W	0	W
00011/00100	0	0	0010	2W	0	2W
00101/00110	0	0	0011	3W	0	3W
00111/01000	0	0	0100	4W	0	4W
01001/01010	0	0	0101	5W	0	5W
01011/01100	0	0	0110	6W	0	6W
01101/01110	0	0	0111	7W	0	7W
01111/10000	0/1	0	1000	8W	0/+8	8W/-8W
10001/10010	1	0	0111	7(~W)	+7	-7W
10011/10100	1	0	0110	6(~W)	+6	-6W
10101/10110	1	0	0101	5(~W)	+5	-5W
10111/11000	1	0	0100	4(~W)	+4	-4W
11001/11010	1	0	0011	3(~W)	+3	-3W
11011/11100	1	0	0010	2(~W)	+2	-2W
11101/11110	1	0	0001	(~W)	+1	-W
11111	1	1	0000	0	0	0

Fig. 6. Mechanism of Radix16 + LUT multiplication and Radix16 truth table.

optimization method to reduce the circuit flips of this part of the circuit. Its main mechanism is detailed in Fig. 6.

In the truth table in Fig. 6, we divide the partial product of the Radix16 multiplication into two types. One type can be realized with only a left shift operation $\{\pm W,\ \pm 2W,\ \pm 4W,\ \pm 8W\}$, whereas the other type requires both a left shift and an addition operation $\{\pm 3W,\ \pm 5W,\ \pm 6W,\ \pm 7W\}$, representing 53.3% of the partial product codes (0 is not included). $\pm 7W$ requires two additional operations. To reduce the high power cost of the addition when encoding partial products, we use the LUT method to implement $\{\pm 3W,\ \pm 5W,\ \pm 6W,\ \pm 7W\}$ partial product encoding. This can reduce the dynamic power consumption by 21.7%.

B. Radix16 + LUT-Based Digital CiM Macro

Fig. 7(a) shows the hardware architecture of the 4 × 32 CiM macro. Each 4 × 32 CiM macro consists of a Radix16 encoder unit, a 4 × 32 SRAM array, a weight encoder, a partial product right shifter (PPRS), and an AT and accumulator unit. Each 4 × 32 SRAM array is assigned four Radix16 encoders in the Radix16 encoder unit.

The 8-bit integer is split into two 5-bit Radix16codes to be input in two cycles [t_0 and t_1 in Fig. 7(a)]. In each calculation cycle, four encoders encode the four-row Radix16code data to give three control signals, NEG, EIGHT[3:0], and ZERO, according to the 5-bit activation data. The signal EIGHT[3:0] is used to represent the absolute value of the weight multiplier, with a maximum value of 8. These control signals are input into the weight encoder. The shifter unit and the LUT unit are integrated into the SRAM array to obtain the P[10:0] of the weight data. These four-row P[10:0] data are input to the PPRS for sparsity optimization, as described below in Section VI-B. The P[10:0] results from the PPRS are sent to the four-input AT and accumulator to obtain four-row sum results. The complement code is generated by the complement code adder and added to the four-row sum to give a complete 18-bit partial product. The CiM macro obtains complete four-row 18-bit MAC calculation results every two calculation cycles. Both the shifter unit and the LUT unit are implemented using customized digital circuits.

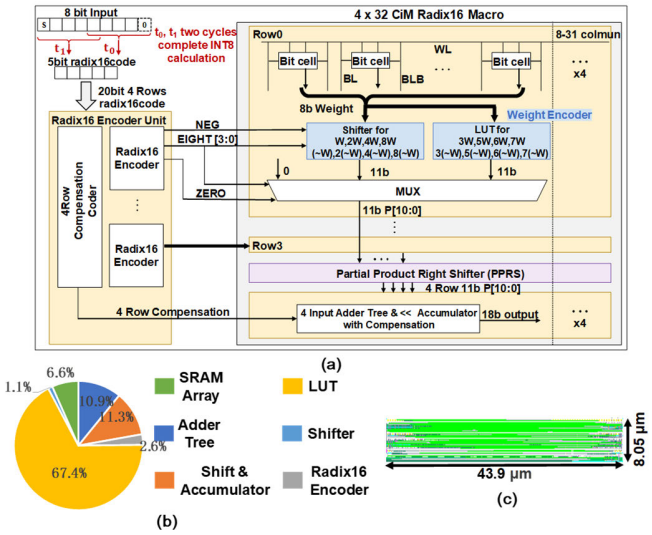


Fig. 7. (a) Hardware scheme for the proposed 4 × 32 CiM macro, (b) 4 × 32 macro area breakdown, and (c) layout of the LUT circuit.

Fig. 7(b) details a breakdown of a single 4 × 32 macro area. In our design, the area of a single 4 × 32 macro is 8400 μm^2. This area data includes all circuits used for calculations in the macro and the routes between the pins of these circuits. Circuits for sparsity techniques are not included. Fig. 7(c) shows the layout of a single LUT circuit. The LUTs are designed in Virtuoso using a method that maximizes the reuse of digital logic units. It is composed of multiple digital gate unit circuits, which are similar to an adder. The layout complexity of the LUT circuit is not high, and all routes can be realized within eight layers of metal, herein, M9 and M10 are used for power rails. Because it is the largest circuit in the macro, the layout of other modules of the macro is placed around the LUT layout.

C. Performance Comparison

Fig. 8 shows a plot of the performance of our Radix16 + LUT scheme and the existing digital CiM macro for comparison. We perform the macro-level evaluation in Virtuoso at [0.9 V, 100 MHz, TT corner, 25 °C]. To ensure a fair comparison, the power consumption of PPRS for sparsity optimization is excluded. Although the Radix16 + LUT method has higher power consumption in each calculation cycle compared to existing schemes, in each complete MAC calculation, the power cost is reduced by 2.44×. The additional circuits introduced to the CiM macro mainly include the Radix16 encoder unit and the weight encoder, which accounts for 7.58% and 16.66% of the MAC power, respectively, in a macrolevel evaluation. The LUT optimization method reduces the MAC power by 21.7%. It can also be seen from Fig. 8 that the reduction in MAC computing power consumption brought by Radix16 technology is very small compared to Radix4 [15]. This proves that the LUT technology is important for further reduction of the computing power consumption in our digital CiM macro design.

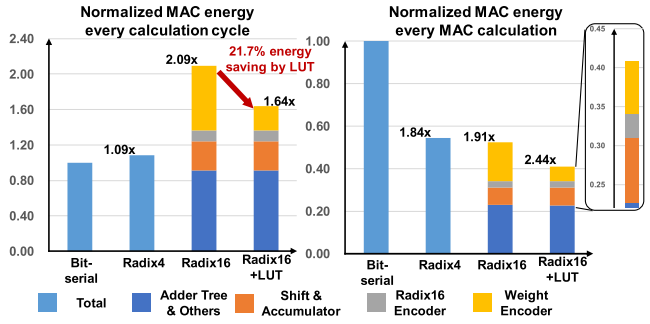


Fig. 8. Macrolevel MAC energy in every calculation cycle. MAC energy in every MAC calculation. All for INT8 × INT8.

V. Configurable Winograd-Domain CONV for Microarchitecture-Level Optimization

A. Hybrid Winograd Algorithm

Previous work [22] on the Winograd algorithm mainly focused on the design of a customized microarchitecture for one of the levels to accelerate the CONV workload. As mentioned in Section II-B, the Winograd algorithm using only the F2 level under INT8 accuracy causes no loss in algorithm accuracy. When only the F4-level or F6-level Winograd algorithm is used, better acceleration effects can be achieved, but the accuracy of the algorithm will drop dramatically. To avoid this problem, we devise a hybrid (i.e., F2 and F4) Winograd algorithm, where the training process determines which level of the Winograd algorithm is chosen and used at which layer of the algorithm, thus reducing the MAC workload as much as possible with an acceptable loss in algorithm accuracy.

The transformation operation of the F6 Winograd algorithm has an excessively high power cost, and the accuracy loss of F6 is too large [23]. For this reason, we only employ F2, F4, and regular spatial-domain CONV operations to optimize the performance of a CiM-based processor at the microarchitecture level. Fig. 9(c) illustrates the principle of operation of the F4 Winograd algorithm on 3 × 3 CONV. The spatial-domain CONV workload is converted into an elementwise matrix multiplication (EWMM) operation in the Winograd domain via a transformation operation, resulting in a four times reduction in MAC workload. The area and power cost of this transformation operation are minimized by customizing the transformation unit. The Winograd-domain CONV dataflow is shown in Fig. 10. The three transformation OUs, $B^T()B$, $G()G^T$, and $A^T()A$, are used to carry out the Winograd transformation operation of F2 or F4 according to the instructions decoded by the top control unit. The input and output buffers are placed to organize the input and output data, respectively. The implementation of the Winograd algorithm in this work is introduced in detail in Section V-B.

B. Workload Conversion and Mapping

The hybrid Winograd algorithm must adapt to CONV workloads with different kernel sizes and different strides. Given this, we propose kernel decomposition and activation/kernel tile segmentation. Fig. 9(a) exemplifies the principle of this

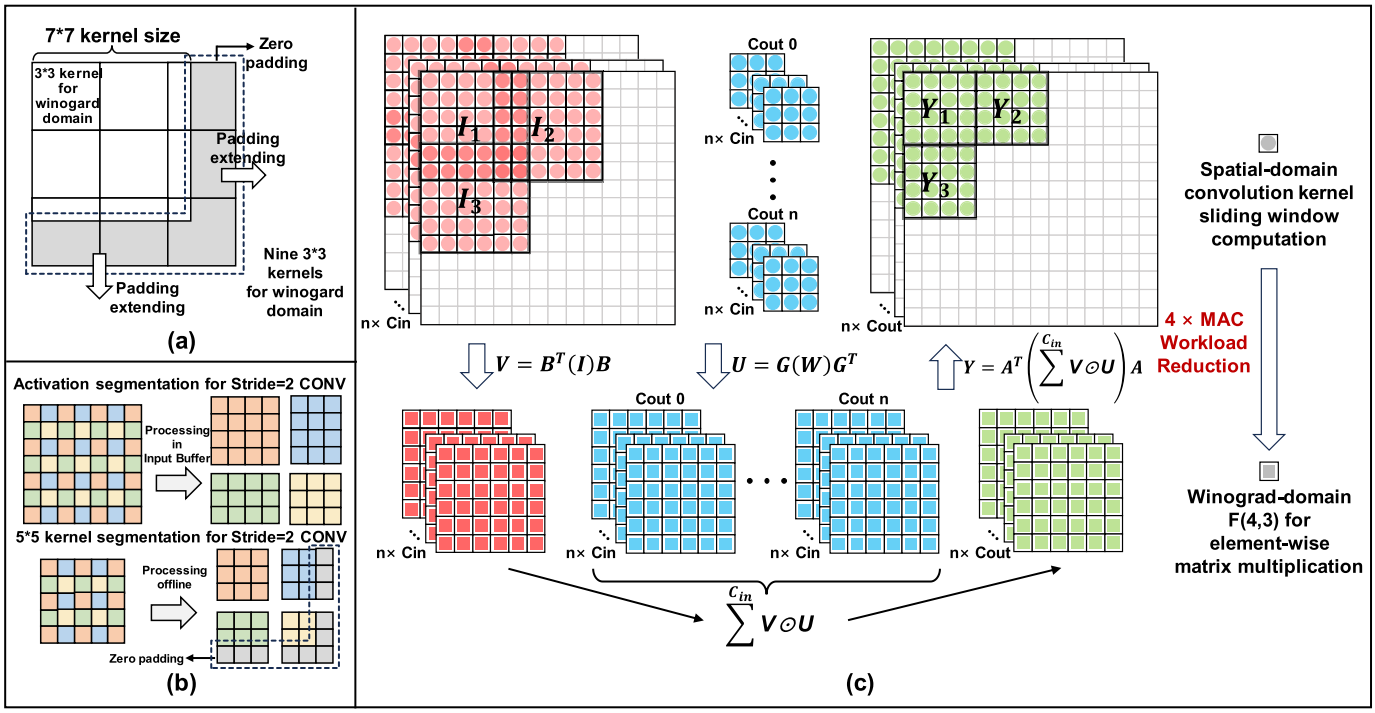


Fig. 9. (a) Example of decomposition for CONV with 7 × 7 kernel, (b) example of activation data segmentation for CONV with stride = 2, and (c) details of F4 [F(4, 3)] Winograd-domain 3 × 3 CONV.

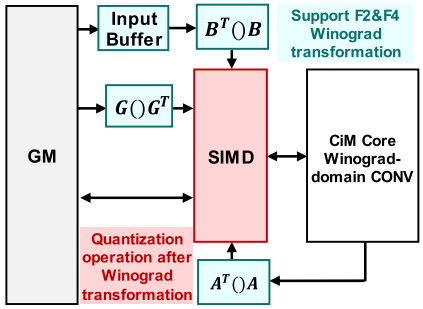


Fig. 10. Winograd-domain CONV dataflow.

method for a CONV workload with stride = 1 and a large (e.g., >3 × 3) kernel. A kernel larger than 3 × 3 can be decomposed into multiple 3 × 3 kernels, to which the Winograd algorithm can be applied at different levels. For a CONV workload where stride = 2, the activation segmentation method is used. Fig. 9(b) shows the implementation of this method with the 5 × 5 kernel, where the activation tile and 5 × 5 kernel are divided into areas of four different colors. The 3 × 3 kernel processes the 3 × 3 CONV workload with stride = 1 in the activation area with the same color, and the workload shown by each color in Fig. 9(b) can be accelerated by the Winograd algorithm. This segmentation method can also be applied to CONV workloads with other kernel sizes (e.g., 3 × 3 or 7 × 7) with stride = 2. In the above operations, the kernel decomposition and segmentation operations are implemented offline, and the activation segmentation operations are executed in the on-chip input buffer.

We exploit a pixel/channel order (PCO) mapping method for efficient mapping of the different levels of the Winograd-domain workload. The CiM macro can accumulate the resulting products of different rows in the same column to perform the inner product operation, which we call as row accumulation function. As shown in Fig. 9, the Winograd-domain CONV needs to execute the EWMM operation. Unlike a normal CONV, this operation does not need to accumulate all the result values in a single kernel. The only addition operation in EWMM is the addition of the results of different input channels (C_{in}) with the same output channel (C_{out}). Thus, in the intramacro, we use channel order mapping (COM) and exploit the row accumulation function of the CiM macro to realize the addition of different C_{in} product results. We use pixel order mapping (POM) in the intermacro to ensure good scalability of parallelism. Fig. 11 illustrates the mapping principle of the Winograd-domain workload for F2 and F4, where COM is used in a single 4 × 32 macro, and POM is used between multiple 4 × 32 macros.

Workloads with larger numbers of C_{in} and C_{out} can achieve higher hardware utilization, as the sizes of C_{in} and C_{out} may be much larger than the size of the macro, meaning that the macro will easily be filled by the workload. In contrast, workloads with small numbers of C_{in} and C_{out} usually have lower hardware utilization, as there is space in the macro that is not filled by the workload. For this reason, we use a smaller 4 × 32 macro as a single CiM macro that can be scheduled in conjunction with the PCO mapping method to achieve higher hardware utilization on different types of workloads. For workloads with small numbers of C_{in} and C_{out}, each macro supports a minimum workload mapping of $4C_{\text{in}}$ and $4C_{\text{out}}$ for a single pixel. The use of multiple 4 × 32 macros can

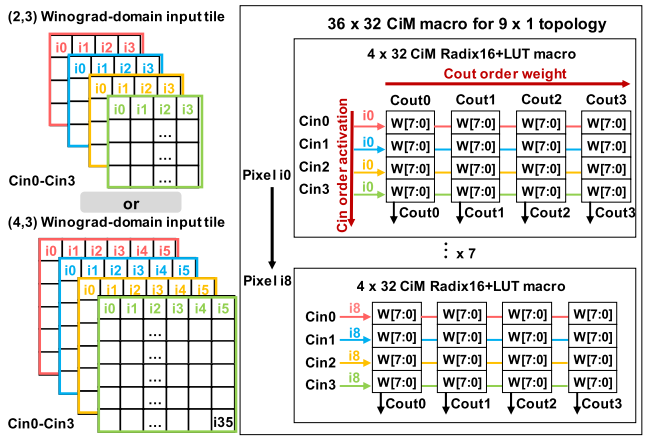


Fig. 11. Pixel/channel-order mapping method for F2&F4 Winograd-domain CONV.

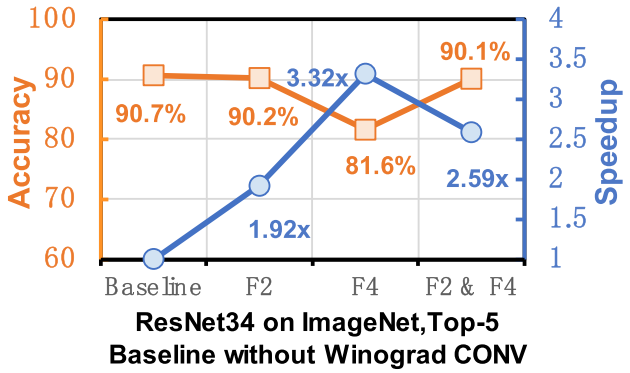


Fig. 12. Accuracy and speedup for the hybrid Winograd algorithm.

enable pixel-level parallelism to improve hardware utilization. For large numbers of C_{in} and C_{out}, where the workload size is much larger than the macro size, high rates of hardware utilization can also be achieved.

C. Accuracy/Speedup Analysis

Fig. 12 shows a plot of the evaluation data for our hybrid Winograd method. For comparison, we conducted four test cases on ResNet34 with the ImageNet dataset. The use of an F2-level Winograd algorithm alone gives higher accuracy, but has an acceleration effect of only 1.92×. Using an F4-level Winograd algorithm alone gives a speedup of 3.32×, but the accuracy of the algorithm drops by 9.1% compared to the baseline, which is unacceptable for most tasks. The hybrid Winograd algorithm proposed in this work uses different levels of optimization at different layers in the network to balance the speedup and accuracy very effectively. Compared with the baseline, we see a performance improvement of 2.59× with a loss of only 0.6% in accuracy. This proves that our hybrid Winograd optimization method can yield high performance with almost no accuracy loss for general CNN workloads. The transformation units required by F2 and F4 are implemented using custom hardwired modules. The area (power) of the F2-level and F4-level transformation circuits accounts for 0.91% (5.33%) and 10.76% (15.97%) of the total, respectively.

VI. MDPS Strategy for Macrolevel Optimization

A. MDPS Strategy

As mentioned in [27], it is very hard to implement unstructured fine-grained sparsity in CiM-based processors due to the rigid array structure of the CiM macro. The application of structured sparsity can alleviate this issue, but this approach is less general, usually requires modifying the loss function for training of the algorithm, and is accompanied by accuracy loss. Structured sparsity with a high sparsity rate will cause the accuracy of the algorithm to drop even further. On the hardware side, the randomly distributed structured sparse data also result in extra computing cycles [12]. To fully utilize the sparsity of dual-side (activation and weight) data in the NN algorithm and to solve the problems raised in previous work, we propose the MDPS strategy. As shown in Fig. 13, for the weight data, we apply a horizontal compression method to improve the utilization of the CiM macro. This compression method changes only the column position of each weight data in the CiM macro, without changing its row position, so it does not introduce reentry of activation data due to the random distribution of sparse data. Sparse weight data will not be stored in CiM macros, meaning that the saved macros can be used to map other workloads (Fig. 13). For the activation data, the proposed strategy strikes a balance between element-level and data block-level sparsity. As shown in Fig. 13, the activation data are converted into compressed activation data and bit-mask data, which are used for sparsity calculation control. Whenever there are four consecutive sparse data in the activation data, the calculation of this part of the data is skipped, corresponding to the input size of the single 4 × 32 macro. The key to the proposed sparsity optimization method is scheduling only at the macrolevel to avoid the problems caused by the inflexibility of the CiM rigid array in the macro. Different macros are processed in parallel.

B. MDPS Scheme

Figs. 14 and 15 demonstrate the architecture and workflow of our MDPS scheme. The sparsity-control microarchitecture consists of three main modules, ISPU, PPRS, and OMB. The bit-mask generator in the ISPU generates bit-mask data for the activation data. The tile compressor and tile allocator are used for processing the noncompressed activation data. Each 36 × 32 macro has a 1 kB OMB for rearranging the MAC results. To explain the proposed sparsity principle in detail, we divide the entire sparsity processing procedure into three parts: 1) activation data processing; 2) right shift accumulation MAC calculation; and 3) merging of the MAC results.

1) As shown in Fig. 14, the uncompressed activation data are first sent to the ISPU, and the bit-mask generator generates bit-mask data based on the sparsity of the activation data for sparsity optimization control. According to the principle described in Section V-A, since calculations of four consecutive sparse data are skipped, this part of the consecutive sparse data in the activation data will be removed by the tile compressor. The remaining activation data will be input into the CiM macro in sequence by the tile allocator for calculation.

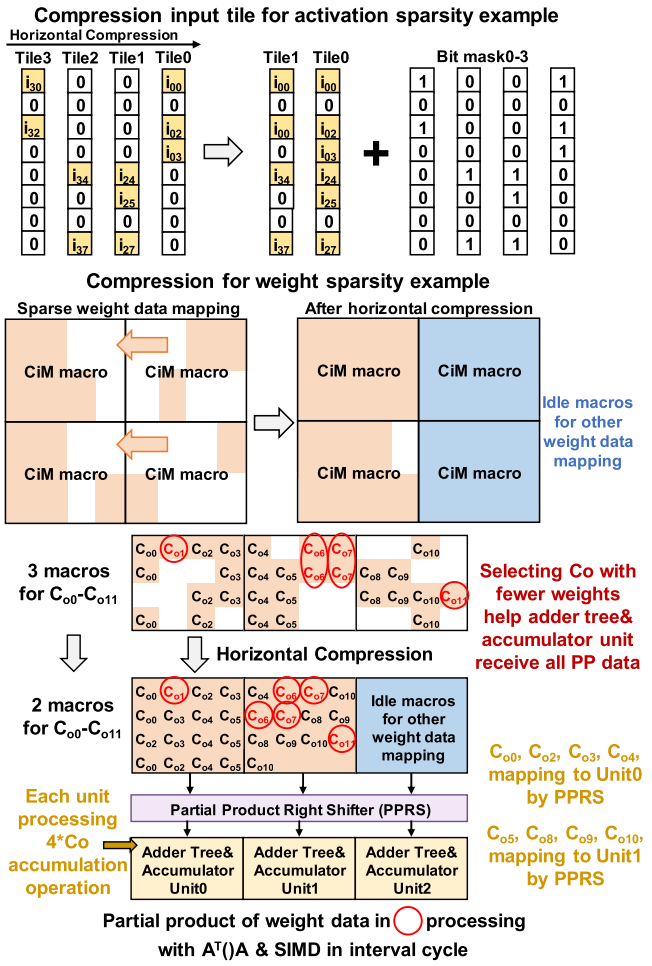


Fig. 13. Procedure of the proposed MDPS strategy.

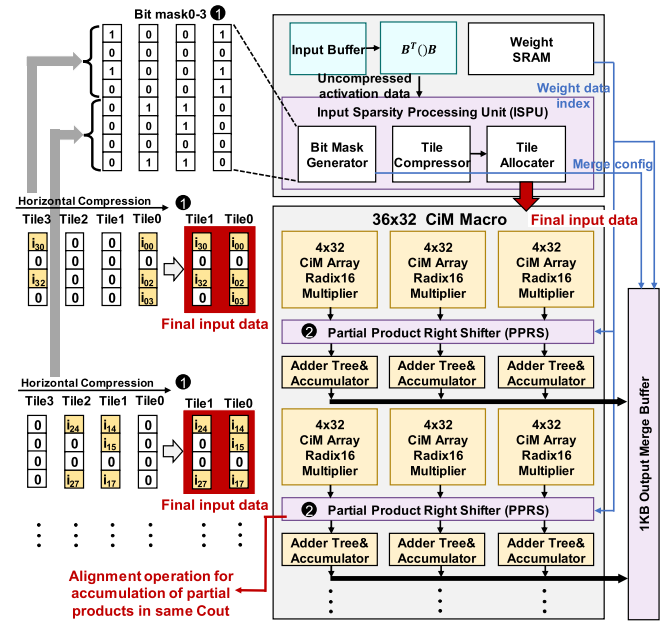


Fig. 14. Architecture and principle of step 1), activation data processing, and step 2), the right shift accumulation MAC calculation, in the proposed MDPS strategy.

2) As described in Section V-A, the weight data stored in the CiM macro will be compressed in the horizontal direction before calculation to improve the utilization of the CiM macro. The direction of weight data compression is to the left, meaning that each P[10:0] in the macro needs to be moved back to the correct position before entering the AT and accumulator so that the accumulation operations of different rows can be calculated correctly. The PPRS will shift the correct P[10:0] value to the input end of the four-input AT based on the weight index data.
3) The first two steps skip the invalid sparse data workloads and only compute the valid dense data. These valid MAC calculation results need to be written to the correct location in the output tile, and this part of the operation is performed in the OMB. As shown in Fig. 15, the weight index and the merge config data generated by the bit-mask data are used to control the writing address of the buffer to achieve the merging operation. The valid MAC calculation result is written to the correct location in the buffer subarray, and the zero-value data are written to the blank location. When all the data in the tile have been calculated, the OMB outputs the complete tile for the subsequent calculations. In step 2) it may happen that the PPRS circuit cannot remap all the data to the AT and accumulator unit because there are too many C_{out} as shown in Fig. 13. These partial product data will be sent directly to the OMB in an interval cycle after the AT and accumulator unit completes the complement code addition operation. As shown in Fig. 15, the tiles from OMB transformed by $A^T()A$ are sent to SIMD in the interval cycle to implement addition calculation. We implement the addition of these partial products using SIMD and data paths between global SRAM because our proposed CiM macro outputs a complete result every two cycles. To achieve latency hiding, we perform these operations in an interval cycle to minimize pauses caused by sparse data.

C. Performance Evaluation

Fig. 16 shows the evaluation of the MDPS strategy. The benchmark was the entire ResNet34 applied to the Top-5 task on the ImageNet dataset. We compressed the weight data of the algorithm with a sparsity rate of 50% through quantification, and selected activation data with a sparsity rate of 50% for testing. It can be seen that the performance improvement brought by the 50% activation/weight data is 1.84×, and 1.76× respectively. The proposed method achieves an improvement of 3.11× in the energy efficiency in the ResNet34 test environment described above. The microarchitectural circuits used for sparsity control were ISPU, PPSR, and OMB, with the extra area and power consumption accounting for 4.5% and 7.8% of the overall totals, respectively. This proves that our developed MDPS method is a very efficient sparsity optimization strategy.

VII. Measurement Results

A. Fabricated Chip and Measurement Platform

Fig. 17 shows the layout of the chip and a performance summary. The proposed CiM-based processor was fabricated

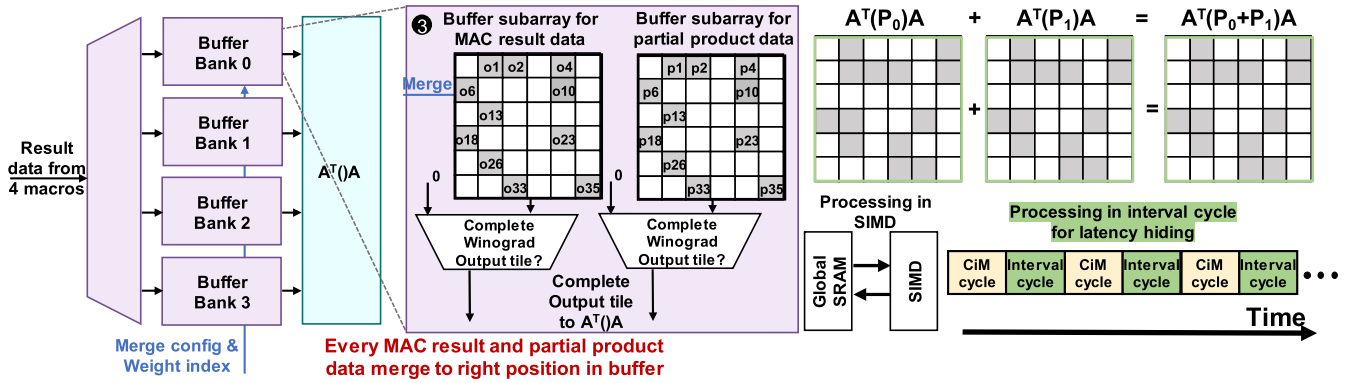


Fig. 15. Principle of operation of step 3), merging of MAC results.

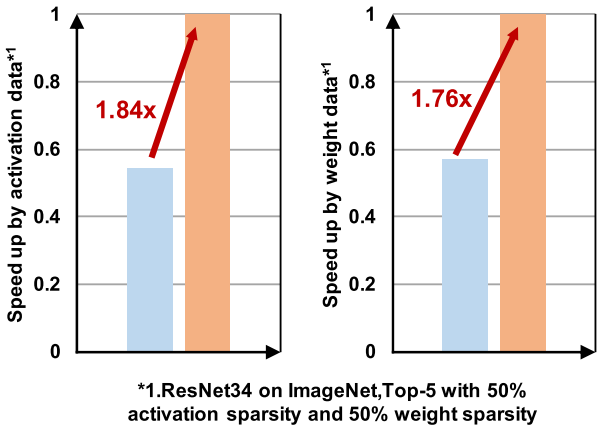


Fig. 16. Evaluation of our MDPS method in terms of speed up by sparse activation/weight data.

TABLE II
OVERALL EVALUATION ON DIFFERENT NN MODELS

Model	ResNet34	ResNet50
Dataset	ImageNet, Top-5	ImageNet, Top-1
Precision (Activation/Weight)	INT8/INT8	INT8/INT8
Baseline Accuracy	90.72%	80.86%
Chip Accuracy	90.16% (0.56% loss)	78.71% (2.15% loss)
Quantization Loss	0.43%	1.18%
Winograd Loss	0.13%	0.97%
Speedup with Winograd CONV	2.59x	1.51x
Speedup with Winograd CONV + Macro-level Sparsity	8.05x	4.85x

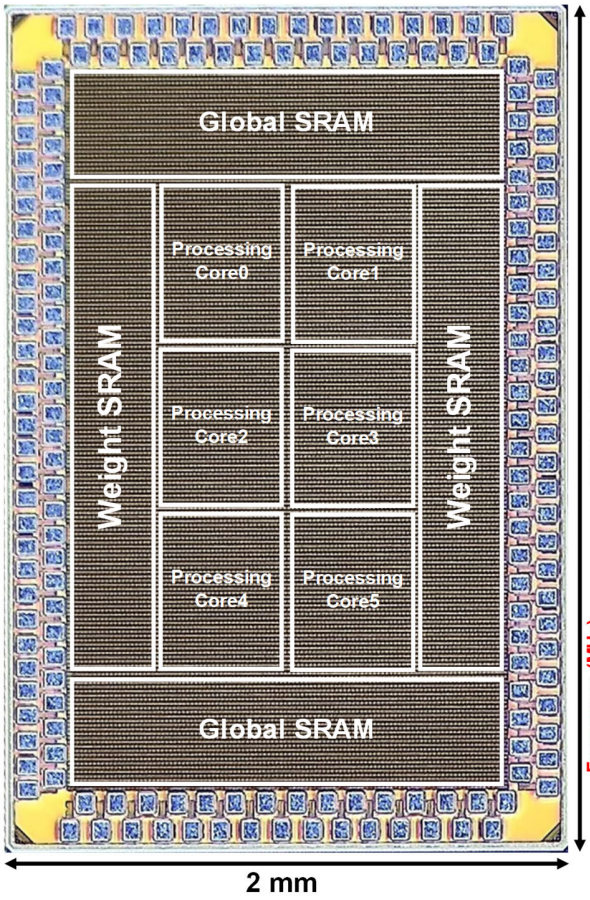
TABLE III
WINOGRAD CONVOLUTION TYPE OF EACH LAYER

Output Size	ResNet34 K*K, Cout, Quantity	Winograd Type	ResNet50 K*K, Cout, Quantity	Winograd Type	Speedup
112*112	7*7, 64, 1	F2	7*7, 64, 1	F2	1.36x
56*56	3*3, 64, 6	F2	3*3, 64, 6	F2	2.25x
28*28	3*3, 128, 8	F4	3*3, 128, 8	F4	4x
14*14	3*3, 256, 12	F4	3*3, 256, 12	F4	
7*7	3*3, 512, 6	F2	3*3, 512, 6	F2	2.25x
1*1	1*1, 1000, 1	NA	1*1, 1000, 1	NA	1x

using 28-nm CMOS technology and occupied a total area of 6.02 mm^2. The test platform of the CiM-based accelerator included a host PC, a Xilinx ZCU-106 field-programmable gate array (FPGA) board, a test printed circuit board (PCB), two FPGA mezzanine connector (FMC) boards, a dc power supply, and an oscilloscope. As illustrated in Fig. 18, our chip was stacked on the test PCB. The oscilloscope was used to track the working status signal of the chip. All the data from the test dataset and weight data for the network were preloaded from the FPGA dynamic random access memory (DRAM) into the on-chip weight SRAM through the FMC board. The FPGA input a test signal to the test chip and performed data transmission, and the calculation result was collected by the FPGA and returned to the host PC for verification purposes. The core voltage of our prototype could be adjusted between 0.6 and 1.1 V when setting the I/O voltage of 2.5 V, and the working frequency ranged from 78 to 287 MHz. The power consumption for all the on-chip digital logic and CiM macro was included, but the power required for the chip I/O and FPGA DRAM was excluded. The peak system energy efficiency reached 258.5 TOPS/W with three features at 0.6 V and 116 MHz.

B. Test Benchmark

To assess the performance of our CiM-based processor, we trained two typical algorithms, ResNet34 and ResNet50, and tested them on our chip on the Top-5 and Top-1 tasks, respectively, using the ImageNet dataset. Table II presents performance evaluation data for these two typical algorithms. Compared with the floating-point baseline results, the accuracy losses of the two algorithms were reduced by 0.65% and 2.15%, respectively. Model quantization is the main source of accuracy loss, and another part of the slight decline comes from the optimization of the Winograd algorithm. In the absence of sparsity optimization, our hybrid Winograd optimization method gives accelerations of 2.59× and 1.51×, respectively. When the two acceleration methods (the hybrid Winograd algorithm and the MDPS strategy) are merged and tested on 50% weight sparsity and 50% activation sparsity, the speedup reaches 8.05× and 4.85×, respectively.



CMOS Technology	28nm
Chip Area	2 x 3.01 mm^2
SRAM Size	292KB (4KB for Inst)
Supply Voltage	0.6-1.1V
Frequency	78-287MHz
Power	12.5-49.9mW
Precision	INT8/INT8 (Act/W)
Sparsity Support	CiM-macro level Parallel Sparsity
System Energy Efficiency*1*2	19.9(B)-258.5(A) TOPS/W (INT8/INT8)

*1. Off-chip/IO power and latency is not included. Time for updating weight data into CiM macro is included.
*2. Highest efficiency point (A) at 0.65 V, 116 MHz with Winograd CONV. 1 operation (OP) = 1 multiplication/addition. Presented OP data are equivalent to the non-sparse models.

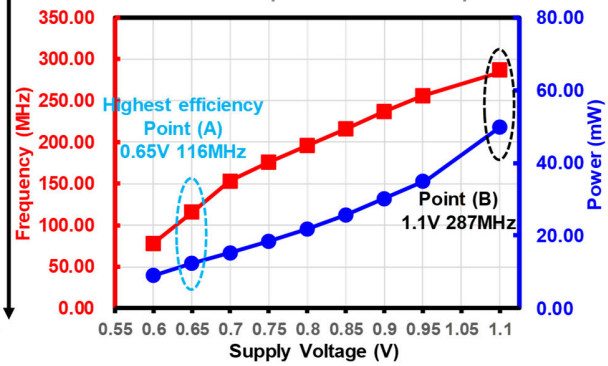


Fig. 17. Chip photograph along with performance summary.

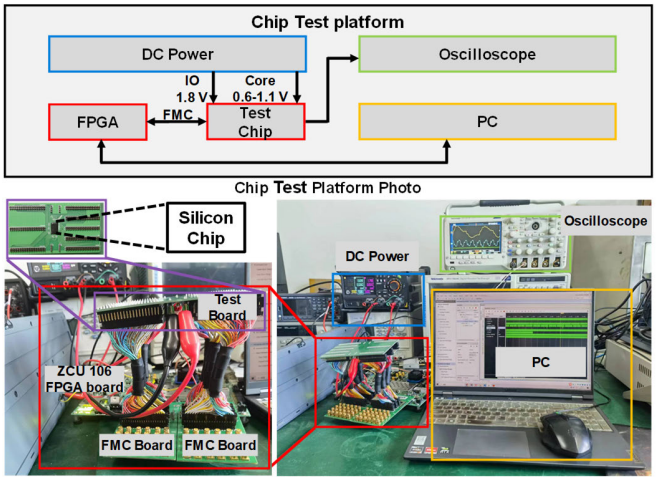


Fig. 18. Test platform of our CiM-based processor chip.

TABLE IV
NORMALIZED PEAK SYSTEM THROUGHPUT OF EVERY REFERENCE

Reference[1]	1/ Number of Cycles	Feature 1	Feature 2	Feature 3	Final Results
ISSCC 2023 [13][2]	0.125	2.69	5.87	NA	1.98
ISSCC 2023 [17][3]	0.125	2.38	2.39	2.67	1.90
ISSCC 2023 [18][4]	0.25	NA	1.88	2.81	1.32
This work	**0.5**	**NA**	**2.59[5]**	**3.11[5]**	**4.03**

1. "NA" means that the feature in the paper does not contribute to throughput. 2. Three features are FP-to-INT CiM workflow, sparse-digital core and low-MAC-value macro. 3. Three features are attention sparsity, token sparsity and bit sparsity. 4. Three features are sparse gathering workflow, epual operation-based CiM initializer and Input-LookAhead CiM. 5. From ResNet34 test.

C. Winograd and Sparsity Contribution

We present the breakdown of the accuracy loss by quantization and also provide information about the Winograd level to which the convolutional layers belong in the algorithms we tested. These data have been supplemented to Tables II and III, respectively. It can be observed that for ResNet50, the accuracy caused by quantization decreases more. The improved Winograd algorithm proposed in our second innovative point does not significantly reduce the accuracy of the algorithm. This is mainly because the Winograd algorithm to which the convolutional layers belong is adjustable. From the data in Table II, it can be seen that the acceleration effect of the Winograd algorithm is more obvious on ResNet34. This is because ResNet34 accounts for a higher proportion of 3×3 convolution calculations at 96.2% compared to ResNet50, while the proportion of 3×3 convolution calculations in ResNet50 is only 47.9%.

TABLE V
COMPARISON WITH STATE-OF-THE-ART DIGITAL CiM-BASED PROCESSORS

		ISSCC 2023 [13]	ISSCC 2022 [15]	ISSCC 2021 [8]	ISSCC 2023 [17]	ISSCC 2023 [18]	**This Work**
Application		Recognition CNN	Recognition CNN	Recognition CNN	Multimodal transformer	GCN & DLRM	**Recognition CNN**
Macro Implementation in Digital CiM		Bit Serial	Radix4	Bit Serial	Bit Serial	Radix4	**Radix16+LUT**
CMOS Process (nm)		28	28	22	28	28	**28**
Supply Voltage (V)		0.47-0.9(System)	0.6-1.0	0.72	0.6–1.0	0.6–1.0	**0.6–1.1**
Chip Area (mm^2)		4.54	6.69	0.202 (macro)	14.36	12.42	**6.02**
Operating Frequency (MHz)		10-400	50-220	500	85–275	115-290	**78–287**
CiM Cycle Number (INT8/INT8)		8	4	8	8	4	**2**
Hybrid Winograd CONV		No	No	No	No	No	**Yes**
Sparsity Support		Digital Sparsity Core	No	No	Attention/Token/Bit Sparsity	Dual-Side Sparsity	**Macro-Level Dual-Side Sparsity**
Peak Reported Performance (TOPS) (INT8/INT8)		2.41	1.35	0.92	3.55	0.89	**7.98**[1]
Normalized Peak System Throughput (INT8/INT8)		1.98	0.25	0.125	1.90	1.32	**4.03**
Peak System Energy Efficiency[2] (TOPS/W) (INT8/INT8)	Worst case[3]	12.8	19.5	24.7 (Macro)	48.4	13.1	19.9[5]
	Best case[4]	75.0	36.5	24.7 (Macro)	101.1	85.0	**258.5**[6]

1.Highest performance at point B (1.1 V, 287 MHz) in Fig. 17. 2. Off-chip/IO power and latency is not included. Power and latency for updating weights into CiM macro are included. 1 operation (OP) = 1 multiplication/addition. OP data presented here are equivalent to non-sparse models. 3. Point B in Fig. 17. Worst reported value for fair comparison. Only Radix16+LUT macro included. The performance improvements brought by hybrid Winograd CONV and MDPS are not included. 4. Point A in Fig. 17. Best reported value for fair comparison. Radix16+LUT macro, hybrid Winograd CONV (2.59x energy efficiency improvement) and MDPS (3.11x energy efficiency improvement) are included. 5. Measured at 1.1 V, 287 MHz. 6. Measured at 0.65 V, 116 MHz.

TABLE VI
COMPARISON OF DIGITAL AND CiM ARCHITECTURE

	Digital	**CiM**
Array Feature	High flexibility	Low flexibility
Data Reuse	No fixed reuse rules	Fixed reuse rules
Energy Efficiency	Low	High
Throughput	High	High (This work)

Previous work on the efficiency of sparsity in the Winograd domain has addressed this issue well at the algorithmic level [29]. It places the weight data pruning after the Winograd transformation instead of before. We use the method in [28] and [29] to get the weight data after Winograd transformation off-chip. Using the above method, the sparsity of the data is not affected by the Winograd transformation. This part of the weight data is stored in the CiM macro in a sparsity compression manner. The sparsity compression and storage operations are implemented by the SIMD unit.

D. Comparison With the State of the Art

In many previous works, the number of computing units and the working frequency were different, which would make comparison difficult throughout. For this reason, we used a new indicator, which is called "normalized peak system throughput." This indicator is used to describe the peak system throughput per MAC unit and MHz. The formula for calculating this indicator in INT8 × INT8 is as follows:

The result after simplifying the above formula is

The simplified formula can be used to fairly compare the throughput performance of different CiM-based processors. From the formula, we can see that this indicator is only related to the number of calculation cycles of CiM macro for INT8 × INT8 and the contribution of other technologies to throughput improvement. Table IV gives the specific calculation data for each reference in Table V of the article. All the improvement coefficients of each technology in the data Table IV for this indicator can be found in the corresponding article. Each coefficient takes the maximum value of the corresponding technology. Among them, 5.87 of Technology 2 in [13] comes from 75/12.8 = 5.87 in the summary table.

Table V compares our design with state-of-the-art CiM-based processors. To ensure a fair comparison of the system energy efficiency, we scale the reported peak energy efficiency of existing schemes from their technologies and supply voltage to 28 nm and 0.6 V. The study in [13] reported an energy-efficient CiM core for handling dense data calculations and a flexible digital core for handling random unstructured sparse data calculations. However, the sizes of the intensive and sparse workloads are different for different algorithms. The workloads processed by two computing cores may be unbalanced, which may result in one core being in an idle state for a long time with low hardware utilization. Tu et al. [15] presented a reconfigurable Radix4-based processor that reduced the number of CiM calculation cycles of the bit-serial method by half. The schemes in [8], [9], and [10] were based only on a digital CiM macro. The processor in [17] exploited attention/token sparsity to specifically accelerate multimodal transformer tasks and introduced bit sparsity as

$$\text{Normalized peak system throughput} = \frac{\text{Operating frequency} * \text{number of MAC units} * 2}{\text{Operating frequency} * \text{number of MAC units} * 2 * \text{number of calculation cycles} * \text{Throughput improvement by other technologies}} \quad (4)$$

$$\text{normalized peak system throughput} = \frac{1}{\text{number of calculation cycles}} * \text{throughput improvement by other technologies} \quad (5)$$

a natural sparsity optimization method for a general workload. We note that bit sparsity optimization can reduce the number of the CiM calculation cycles, but its reduction rate is determined by the sparsity rate of the activation data. In [18], a processor was designed to target the GCN and DLRM tasks with a sparse gather mode and sparse algebra mode. In this work, we propose a combination of a Radix16 + LUT and hybrid Winograd CONV algorithm as a universal optimization method for general workloads at the circuit level and microarchitecture level, respectively, which can greatly enhance the throughput of a CiM-based processor while maximizing the energy efficiency of the computing system. The three contributions of this article relate to optimizing the performance of the CiM architecture for general CONV workload and GEMM calculations. Hence, the three proposed methods do not conflict with the unique sparsity optimization method of the NN algorithm (e.g., attention/token sparsity in a transformer), and the two are hierarchically decoupled. Our CiM-based processor achieves a peak system energy efficiency of 258.5-TOPS/W in the best case for INT8, representing an increase of 2.55–7.08× compared with prior art.

VIII. Conclusion and Discussion

This article proposed a CiM-based processor to overcome the issue of finding a trade-off between energy efficiency and throughput at the same frequency. We devised universal improvement strategies at three levels. At the circuit level, a Radix16 + LUT-based digital CiM architecture was used to realize two-cycle CiM macros. At the microarchitecture level, a hybrid Winograd CONV microarchitecture, and dataflow were designed to reduce the workload with almost no loss of accuracy. At the data level, a new macrolevel parallel dual-side sparsity strategy was devised for sparse data in NNs. When used with different NNs, such as transformer or GCN/DLRM, our three methods can still achieve general performance improvements at three levels. These three techniques are independent of each other and do not interfere with each other. Every technique can be applied as a general-purpose optimization technique at a specific level in the processor design field. Herein, evaluation data for each technology proves this layer-decoupling method can help processors improve their performance at each level efficiently.

The following Table VI summarizes the characteristics of the traditional digital architectures and CiM architectures. This work strives to devise universal technologies at different levels to help CiM maintain its high computing energy efficiency and make its throughput gradually catch up with the traditional digital scheme. Hopefully, the techniques proposed herein can further assist the CiM architecture in its journey toward universality. A 28-nm prototype approached the best system energy efficiency of 258.5 TOPS/W at 0.65 V and 116 MHz.

References

[1] Y. Wang, C. Zhang, Z. Xie, C. Guo, Y. Liu, and J. Leng, "Dual-side sparse tensor core," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Valencia, Spain, 2021, pp. 1083–1095.

[2] Y. Zhao et al., "Cambricon-Q: A hybrid architecture for efficient training," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Valencia, Spain, Jun. 2021, pp. 706–719.

[3] J. -W. Jang, "Sparsity-aware and re-configurable NPU architecture for Samsung flagship mobile SoC," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Valencia, Spain, 2021, pp. 15–28.

[4] N. P. Jouppi et al., "Ten lessons from three generations shaped Google's TPUv4i : Industrial product," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Valencia, Spain, Jun. 2021, pp. 1–14.

[5] Y. Wang et al., "Trainer: An energy-efficient edge-device training processor supporting dynamic weight pruning," *IEEE J. Solid-State Circuits*, vol. 57, no. 10, pp. 3164–3178, Oct. 2022.

[6] Y. Wang et al., "An energy-efficient transformer processor exploiting dynamic weak relevances in global attention," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 227–242, Jan. 2023.

[7] N. Verma et al., "In-memory computing: Advances and prospects," *IEEE Solid-State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, Aug. 2019.

[8] Y. -D. Chih et al., "16.4 An 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *Proc. IEEE Int. Solid-State Circuits Conf.*, 2021, pp. 252–254.

[9] C.-F. Lee et al., "A 12 nm 121-TOPS/W 41.6-TOPS/mm^2 all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Honolulu, HI, USA, Jun. 2022, pp. 24–25.

[10] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3.

[11] J. Yue et al., "14.3 A 65 nm computing-in-memory-based CNN processor with 2.9-to-35.8TOPS/W system energy efficiency using dynamic-sparsity performance-scaling architecture and energy-efficient inter/intra-macro data reuse," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2020, pp. 234–236.

[12] J. Yue et al., "15.2 A 2.75-to-75.9TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, San Francisco, CA, USA, Feb. 2021, pp. 238–240.

[13] J. Yue et al., "A 28 nm 16.9–300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 1–3.

[14] J. Yue et al., "A 5.6–89.9TOPS/W heterogeneous computing-in-memory SoC with high-utilization producer-consumer architecture and high-frequency read-free CIM macro," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Kyoto, Japan, Jun. 2023, pp. 1–2.

[15] F. Tu et al., "A 28 nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3.

[16] F. Tu et al., "A 28 nm 15.59μJ/token full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 466–468.

[17] F. Tu et al., "16.1 MuITCIM: A 28nm 2.24 μJ/token attention-token-bit hybrid sparse digital CIM-based accelerator for multimodal transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 248–250.

[18] F. Tu et al., "16.4 TensorCIM: A 28 nm 3.7nJ/gather and 8.3TFLOPS/W FP32 digital-CIM tensor processor for MCM-CIM-based beyond-NN acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 254–256.

[19] H. Zhu et al., "COMB-MCM: Computing-on-memory-boundary NN processor with bipolar bitwise sparsity optimization for scalable multi-chiplet-module edge machine learning," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3.

[20] T.-H. Wen et al., "A 28 nm nonvolatile AI edge processor using 4Mb analog-based near-memory-compute ReRAM with 27.2 TOPS/W for tiny AI edge devices," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Kyoto, Japan Jun. 2023, pp. 1–2.

[21] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 4013–4021.

[22] C. Yang, Y. Wang, X. Wang, and L. Geng, "WRA: A 2.2-to-6.3 Tops highly unified dynamically reconfigurable accelerator using a novel Winograd decomposition algorithm for convolutional neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 9, pp. 3480–3493, Sep. 2019.

[23] V. Chikin and V. Kryzhanovskiy, "Channel balancing for accurate quantization of Winograd convolutions," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, New Orleans, LA, USA, Jun. 2022, pp. 12497–12506.

[24] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 1135–1143.

[25] H. Mao et al., "Exploring the granularity of sparsity in convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Honolulu, HI, USA, Jul. 2017, pp. 1927–1934.

[26] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, "Learning structured sparsity in deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 1–9.

[27] F. Tu et al., "SDP: Co-designing algorithm, dataflow, and architecture for in-SRAM sparse NN acceleration," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 1, pp. 109–121, Jan. 2023.

[28] J. Fernandez-Marques, P. Whatmough, A. Mundy, and M. Mattina, "Searching for winograd-aware quantized networks," in *Proc. Mach. Learn. Syst.*, vol. 2, 2020, pp. 14–29.

[29] X. Liu, J. Pool, S. Han, and W. J. Dally, "Efficient sparse-winograd convolutional neural networks," 2018, *arXiv:1802.06367*.

[30] R. Andri, B. Bussolino, A. Cipolletta, L. Cavigelli, and Z. Wang, "Going further with Winograd convolutions: Tap-wise quantization for efficient inference on 4×4 tiles," in *Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Chicago, IL, USA, Oct. 2022, pp. 582–598.

[31] Y. Wang, F. Tu, L. Liu, S. Wei, Y. Xie, and S. Yin, "SPCIM: Sparsity-balanced practical CIM accelerator with optimized spatial–temporal multi-macro utilization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 1, pp. 214–227, Jan. 2023.

[32] C.-Y. Tsai, C.-F. Nien, T.-C. Yu, H.-Y. Yeh, and H.-Y. Cheng, "RePIM: Joint exploitation of activation and weight repetitions for in-ReRAM DNN acceleration," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Dec. 2021, pp. 589–594.

[33] T.-H. Yang et al., "Sparse ReRAM engine: Joint exploration of activation and weight sparsity in compressed neural networks," in *Proc. ACM/IEEE 46th Annu. Int. Symp. Comput. Archit. (ISCA)*, Phoenix, AZ, USA, Jun. 2019, pp. 236–249.

[34] J. H. Shin et al., "Griffin: Rethinking sparse optimization for deep learning architectures," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit. (HPCA)*, Seoul, South Korea, Apr. 2022, pp. 861–875.

[35] H. Wu et al., "A 28-nm computing-in-memory-based super-resolution accelerator incorporating macro-level pipeline and texture/algebraic sparsity," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 2, pp. 689–702, Feb. 2024.

[36] Y. Yuan et al., "34.6 A 28 nm 72.12TFLOPS/W hybrid-domain outer-product based floating-point SRAM computing-in-memory macro with logarithm bit-width residual ADC," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 576–578.

Hao Wu (Member, IEEE) received the B.S. degree in integrated circuits and integrated systems from Xidian University, Xi'an, China. He is currently pursuing the Ph.D. degree with the Key Laboratory of Microelectronics Device and Integrated Technology, Institute of Microelectronics of Chinese Academy of Sciences, Beijing, China.

His research interests include static random access memory (SRAM)-based in-memory computing processor and algorithm-architecture co-design.

Yong Chen (Senior Member, IEEE) received the B.Eng. degree in electronic and information engineering from the Communication University of China (CUC), Beijing, China, in 2005, and the Ph.D. degree in microelectronics and solid-state electronics engineering from the Institute of Microelectronics of Chinese Academy of Sciences (IMECAS), Beijing, in 2010.

From 2010 to 2013, he worked as a Post-Doctoral Researcher with the Institute of Microelectronics, Tsinghua University, Beijing. From 2013 to 2016, he was a Research Fellow with VIRTUS/EEE, Nanyang Technological University, Singapore. Then, he was an Assistant Professor from 2016 to 2022 and an Associate Professor from 2022 to 2024 with the State Key Laboratory of Analog and Mixed-Signal VLSI (AMSV), University of Macau, Macau, China. In 2024, he joined the Department of Electronic Engineering, Tsinghua University, where he is currently an Associate Professor. His research interests include integrated circuit designs involving analog/mixed-signal/RF/mm-wave/sub-THz/wireline.

Dr. Chen was a recipient of the "Haixi" (three places across the Straits) Postgraduate Integrated Circuit Design Competition (Second Prize) in 2009, a co-recipient of the Best Paper Award at the IEEE Asia Pacific Conference on Circuits and Systems (APCCAS) in 2019, the Best Student Paper Award (Third Place) at the IEEE Radio Frequency Integrated Circuits (RFIC) Symposium in 2021, the Macao Science and Technology Invention Award (First Prize) in 2020, and the Macao Science and Technology Invention Award (Second Prize) in 2022, and the Gold Leaf Prize (Top 10% Papers) at the 18th International Conference on Ph.D. Research in Microelectronics and Electronics (PRIME) in 2023. He was recognized as the top five associate editors in 2020, the five highest-performing associate editors in 2021, one of the three best associate editors in 2022, one of the three best reviewers of IEEE TRANSACTION ON VERY LARGE SCALE INTEGRATION SYSTEMS in 2022, and one of the inaugural SSCS reviewer rewards of the IEEE Solid-State Circuits Society (SSCS) in 2023. He serves as an Associate Editor for IEEE TRANSACTION ON VERY LARGE SCALE INTEGRATION SYSTEMS since 2019, a Subject Editor (2022) for Circuits and Systems of *IET Electronics Letters* and an Associate Editor (2020–2023) of *IET Electronics Letters*, an Editor of *International Journal of Circuit Theory and Applications (IJCTA)* since 2020, a Guest Editor (2022–2023) of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS in 2022, and a Guest Editor (2021–2022) of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS II: EXPRESS BRIEFS. He serves as a Vice-Chair (2019–2021) and a Chair (2021–2023) of IEEE Macau CAS Chapter and the Technical Program Committees of several international conferences.

Yiyang Yuan received the B.S. degree in electronic and information engineering from Beijing Institute of Technology, Beijing, China, in 2021. He is currently pursuing the Ph.D. degree with the Institute of Microelectronics of Chinese Academy of Sciences, Beijing.

His research interests include compute-in-memory, resistive random access memory, artificial intelligence, and brain–machine interface.

Jinshan Yue (Member, IEEE) received the B.S. and Ph.D. degrees from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2016 and 2021, respectively.

He is currently a Post-Doctoral Researcher and an Assistant Researcher with the Institute of Microelectronics of Chinese Academy of Sciences, Beijing. His current research interests include energy-efficient neural network processor, nonvolatile memory, and computing-in-memory systems.

Dr. Yue served as a TPC Member in IEEE A-SSCC2023 and ICCAD2023.

Xinghua Wang (Member, IEEE) received the B.Eng. and Ph.D. degrees from Beijing Institute of Technology (BIT), Beijing, China, in 2004 and 2010, respectively.

She is currently an Associate Professor at the School of Integrated Circuits and Electronics, BIT. Her research interests include analog, RF, and computing-in-memory.

Xiaoran Li (Member, IEEE) received the B.Eng. degree in telecommunication engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2011, and the M.E. and Ph.D. degrees in electronics science and technology from Beijing Institute of Technology, Beijing, in 2013 and 2017, respectively.

She was a Visiting Scholar with the Department of Electrical and Computer, Duke University, Durham, NC, USA, from 2015 to 2016. She was a Post-Doctoral Research Fellow with Beijing Institute of Technology, where she is currently an Assistant Professor. Her research interests include analog, radio frequency, millimeter-wave integrated circuits design, computing in memory, sparse sensing, and communications.

Feng Zhang (Senior Member, IEEE) received the B.S. degree from Beijing Institute of Technology, Beijing, China, in 2000, and the M.S. and Ph.D. degrees from the Institute of Microelectronics of Chinese Academy of Sciences (IMECAS), Beijing, in 2002 and 2005, respectively.

In 2010, he joined IMECAS as a Professor. He has published over 80 international journal articles and conference papers. His research interests include memory, low-power digital circuits, related hardware security techniques when applied to the "Smart Internet of Things (IoT)" field.

Dr. Zhang serves as a Technical Committee Member of the IEEE Circuits and Systems Society (CASS).